

Documentación

-Sumador-Restador binario:

Tabla de verdad de semi sumador (Tomado del libro, página 119):

Tabla 4-3
Semisumador

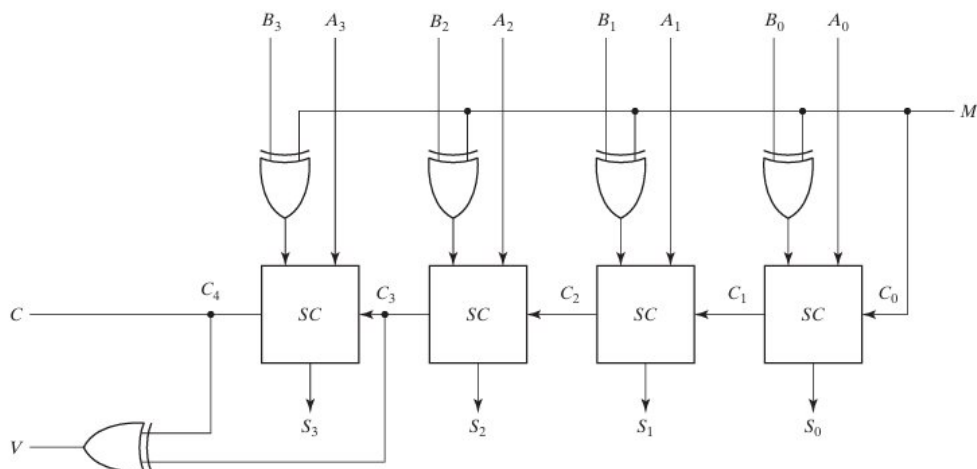
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Tabla de verdad de sumador completo (Página 120 del libro):

Tabla 4-4
Sumador completo

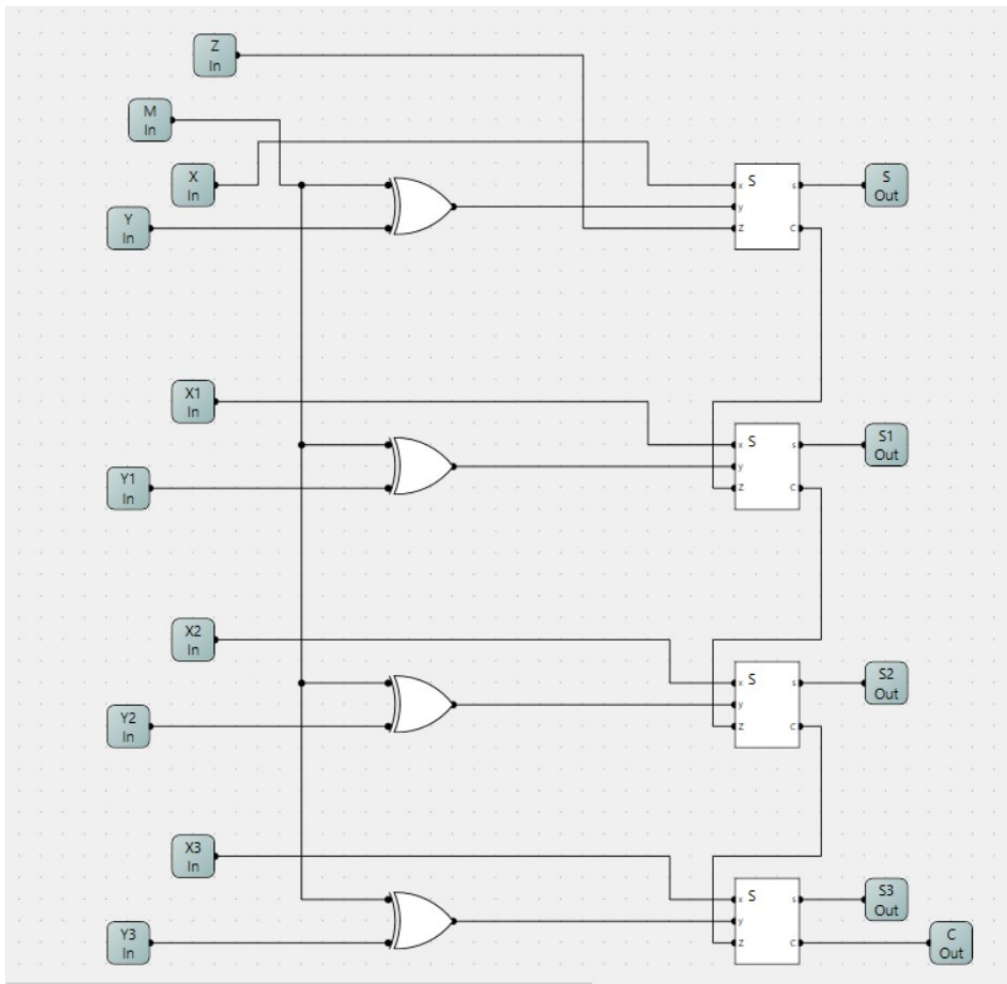
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

-Diagrama de sumador-restador de 4 bits (página 127 del libro):



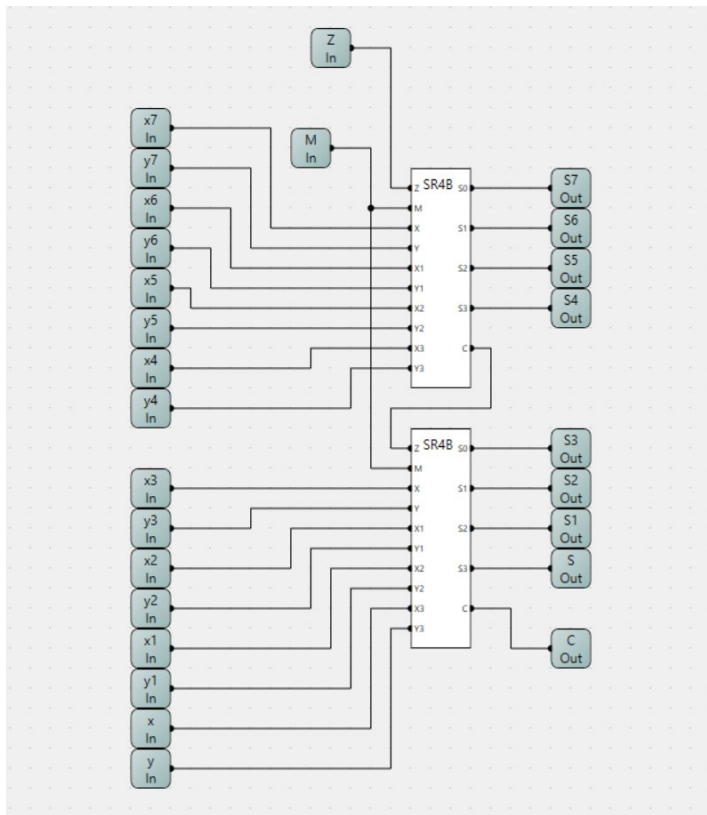
Este sumador-restador de 4 bits utiliza 4 sumadores completos conectados en cascada. Sus entradas son los 8 bits a sumar (4 del sumando y 4 del sumador). Utiliza las compuertas xor y la entrada M (modo) para dirigir que operación realizará. Si M es 0, entonces suma con normalidad, ya que las compuertas xor no afectarán a las entradas B y no se aplicará complemento. Si $M = 1$, las compuertas xor invertirán los bits de las entradas B, generando el complemento a 1, además, el acarreo del primer sumador empezará en 1, lo que aplicará el complemento a 2 (Sumar 1 luego de invertir los bits), realizando así una resta que utiliza las sumas binarias y complementos a dos.

-Sumador-Restador de 4 bits (propio):



Nuestra variante propia agrega un nuevo pin de entrada z (acarreo), porque para el de 8 bits había ciertos conflictos derivados de tener acarreo y modo juntos, esto se explica a continuación.

Sumador-Restador de 8 bits (propio):



Este sumador-restador de 8 bits utiliza dos de 4 bits conectados en cascada. Para evinecta al sumador-restador inicial. Si usáramos el modelo de 4 bits del libro, no podríamos escalar correctamente a 8 bits, ya que si usáramos modo para controlar el acarreo de ambos sumadores-restadores, si hiciéramos una resta, el modo estar errores, se creo una entrada para que modo (m) y acarreo (z) entraran por separado. Modo indicará a ambos circuitos que operación harán. Y acarreo, que se coarúa condicionando todo, ya que controlaría el acarreo y modo juntos, pero para el segundo semi sumador modo y acarreo pueden tener valores distintos, lo que podría generar conflictos. Por eso tomamos a decisión de separar acarreo y modo.

Multiplicador de 3 por 8 bits binario

La multiplicación en binario funciona prácticamente igual que en decimal. Básicamente, vamos multiplicando cada bit del número de abajo (multiplicador) por todos los del número de arriba (multiplicando). Luego sumamos todos esos resultados parciales, desplazándolos un espacio a la izquierda en cada paso. Multiplicar dos bits es exactamente lo mismo que pasarlos por una compuerta AND, ya que esta compuerta solo da 1 si ambos bits son 1.

Para armar cualquier multiplicador de este tipo, se usan unas ecuaciones específicas. Si nuestro multiplicador tiene **J** bits y el multiplicando tiene **K** bits, entonces vamos a necesitar:

- **(J × K)** compuertas AND para sacar todos los productos parciales.
- **(J - 1)** sumadores de K bits para ir sumando esos productos.
- Y el resultado final siempre tendrá **(J + K)** bits.

- Multiplicador de 2x2 bits:

	B_1	B_0	
	A_1	A_0	
	A_0B_1	A_0B_0	
A_1B_1	A_1B_0		
C_3	C_2	C_1	C_0

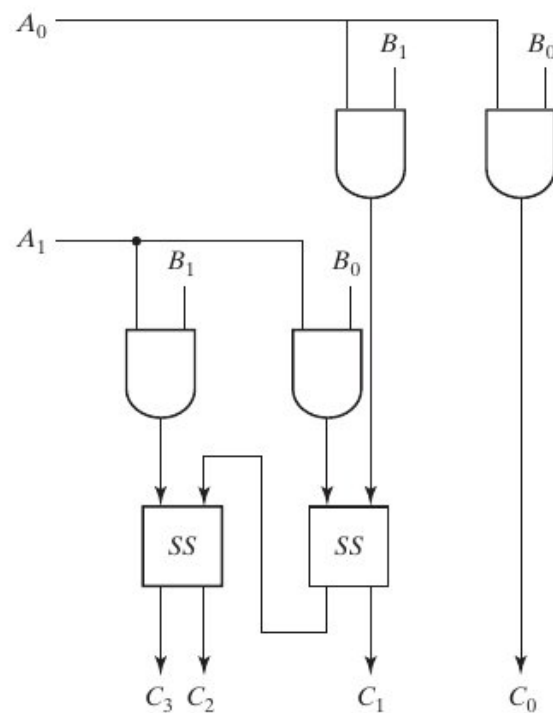


FIGURA 4-15
Multiplicador binario de dos bits por dos bits

Este multiplicador toma 2 bits del multiplicando (B_1, B_0) y 2 del multiplicador (A_1, A_0). Siguiendo las ecuaciones (2×2), usamos exactamente 4 compuertas AND.

El primer producto se saca multiplicando A_0 por B_1 y B_0 . Como detalle de diseño, ese primer bit que sale a la derecha (A_0B_0) baja directo al resultado final (C_0) sin pasar por ningún sumador. Luego multiplicamos A_1 por B_1 y B_0 para el segundo nivel, desplazamos el resultado una posición y lo sumamos con el anterior usando dos semisumadores (SS). Al final, obtenemos nuestro producto de 4 bits.

- Multiplicador de 4x3 bits:

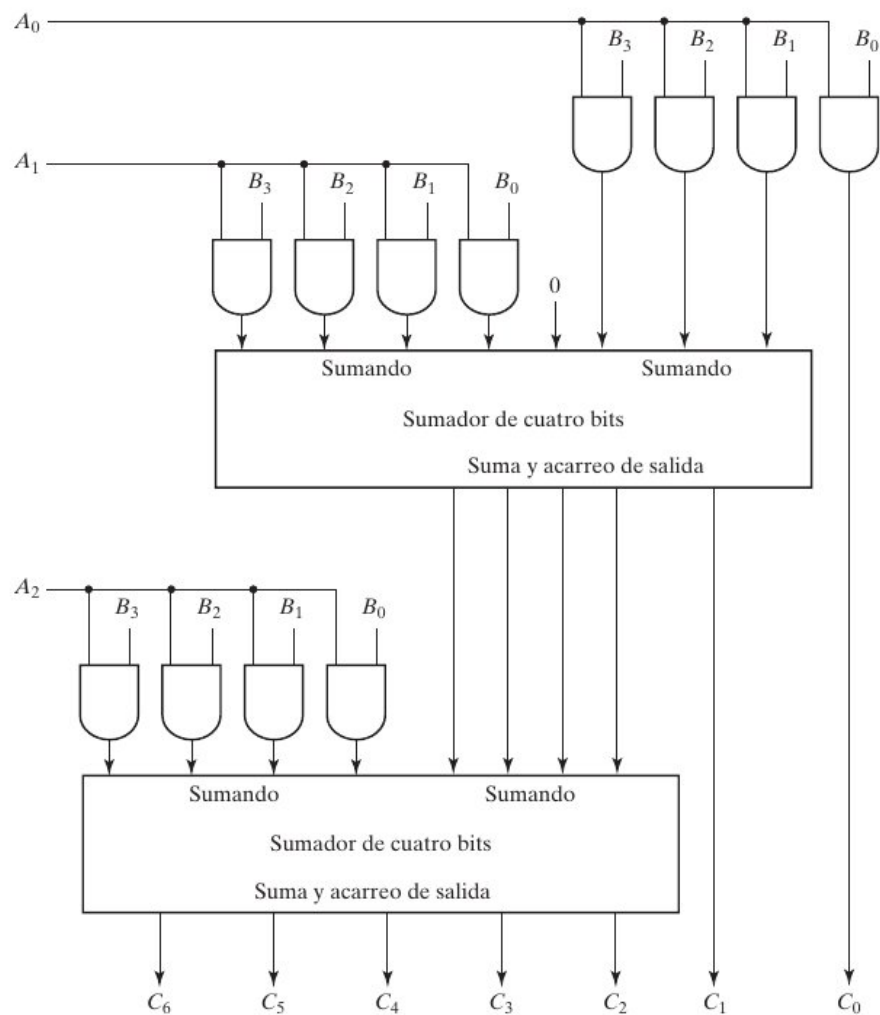
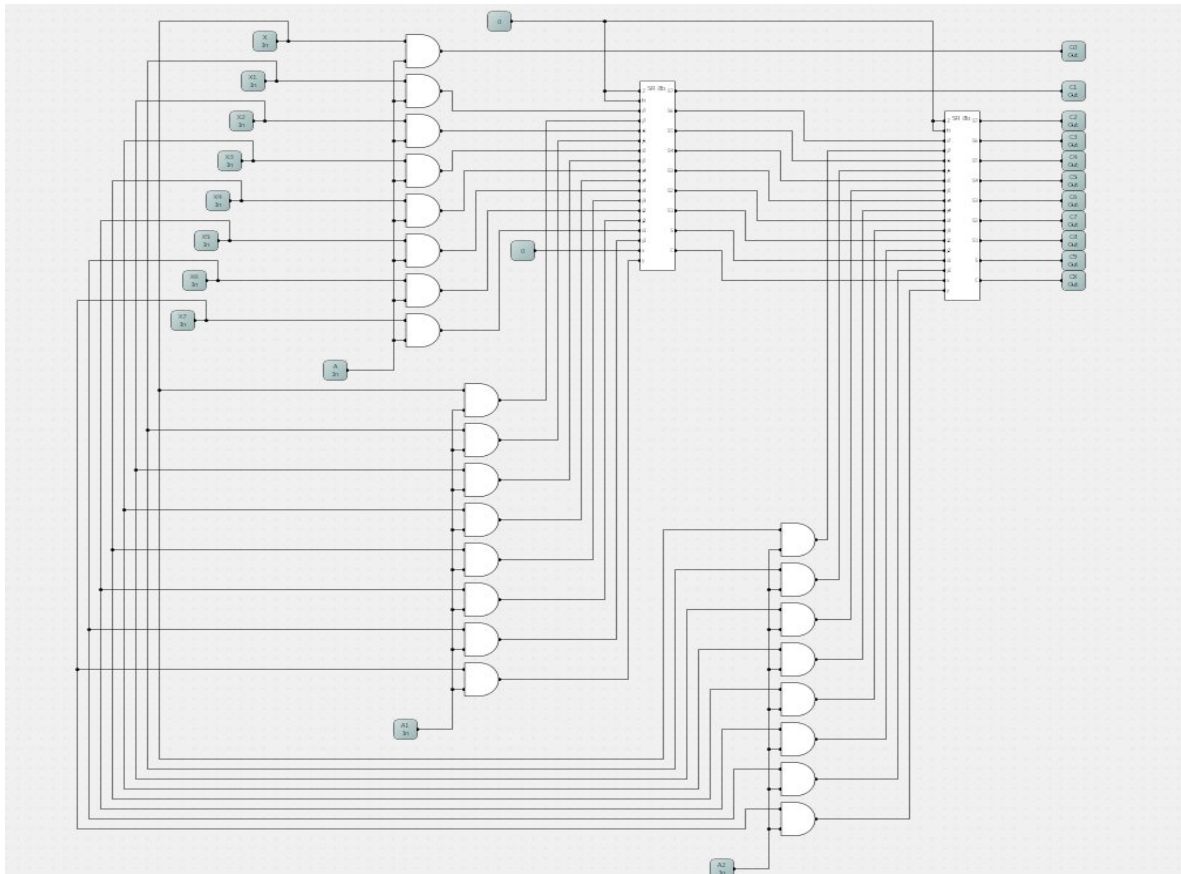


FIGURA 4-16
Multiplicador binario de 4 bits por 3 bits

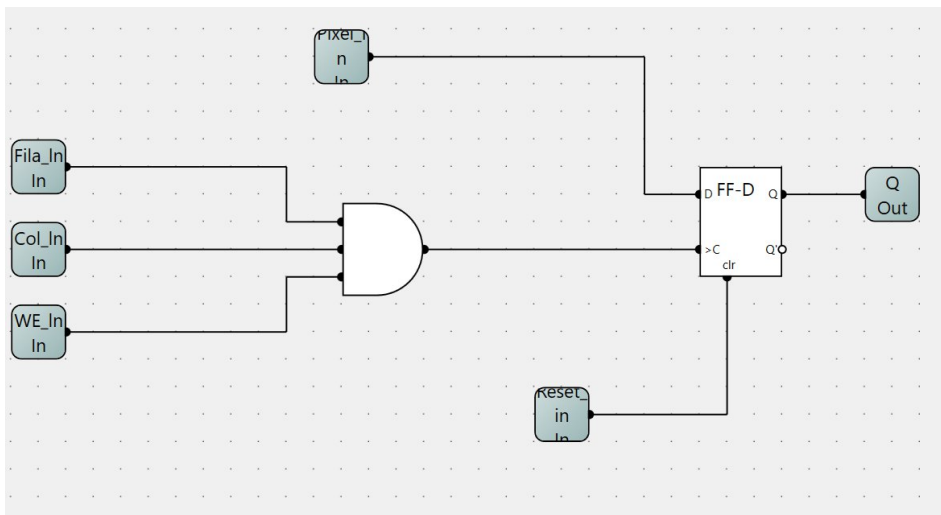
Los principios de este multiplicador son exactamente los mismos, solo debemos adaptar las ecuaciones a la cantidad de bits y seguir el tema de ir corriendo la suma “hacia la izquierda”.

-Multiplicador de 3x8 bits (propio):



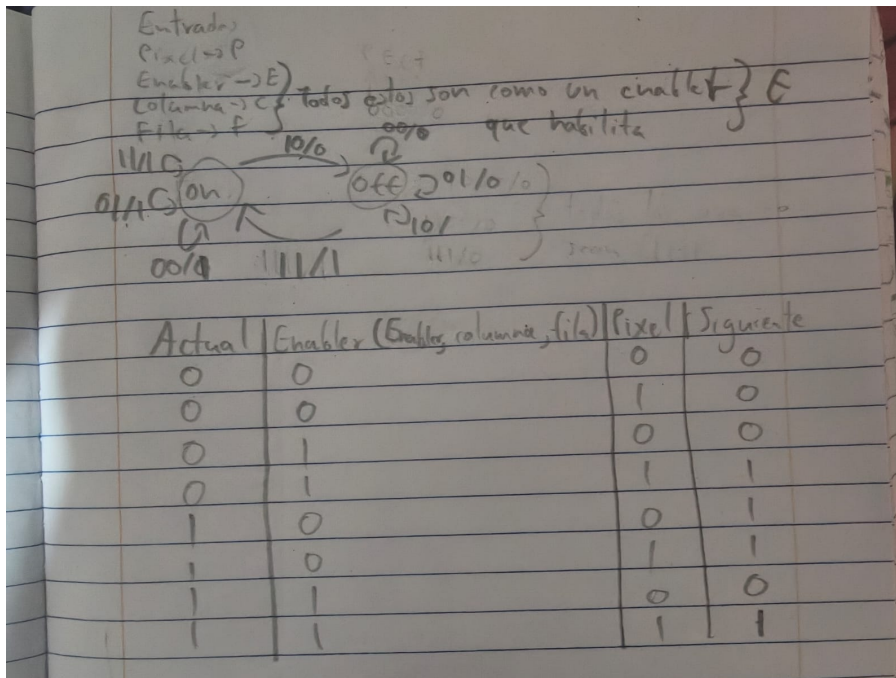
Aquí usamos sumadores restadores de 8 bits. Para que no haya conflictos raros, las entradas de modo/acarreo de estos bloques grandes están conectadas a 0, asegurando que se comporten netamente como sumadores y no intenten restar nada. Al final de todo el recorrido, las salidas nos entregan un súper producto de 11 bits en total (8+3).

Celda individual:

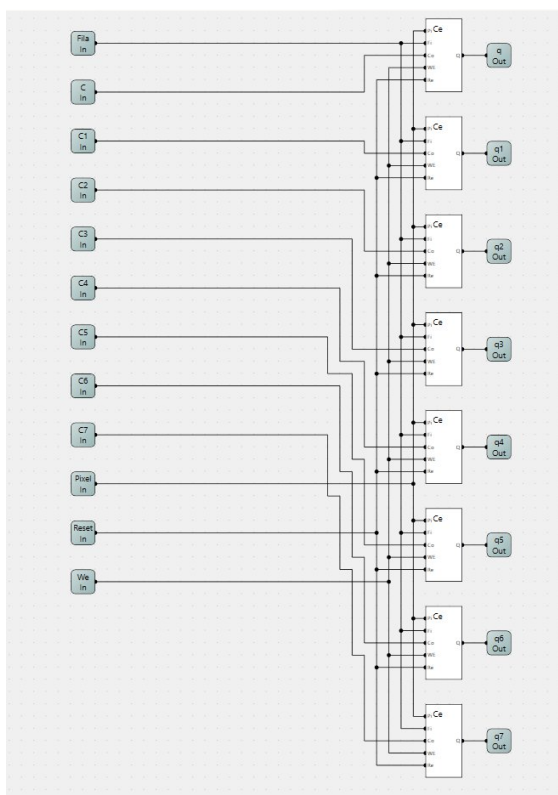


Esta celda es la unidad base de la pantalla que diseñamos, controla un pixel específico de los 64 que están en la matriz. El Flip Flop D almacena el estado del pixel, y la compuerta AND controla cuando debe de escribir. Para que el ff guarde el estado del pixel (Básicamente si enciende o no enciende ese espacio de la matriz). Las entradas de la AND son un enable y la señal de la fila y la columna específica. O sea, el ff no cambiará el dato que guarda (0 o 1) a menos que esté habilitado y reciba la señal de que ese pixel debe encenderse en esa columna y esa fila en específico. Si la AND no deja entrar, el ff mantiene su estado anterior y no cambia.

Diagrama y tabla de la celda:



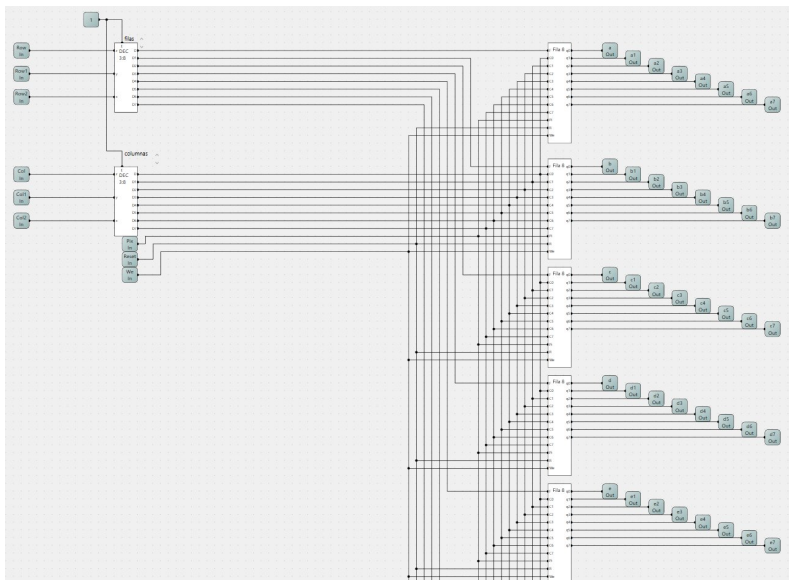
Controlador para cada fila:

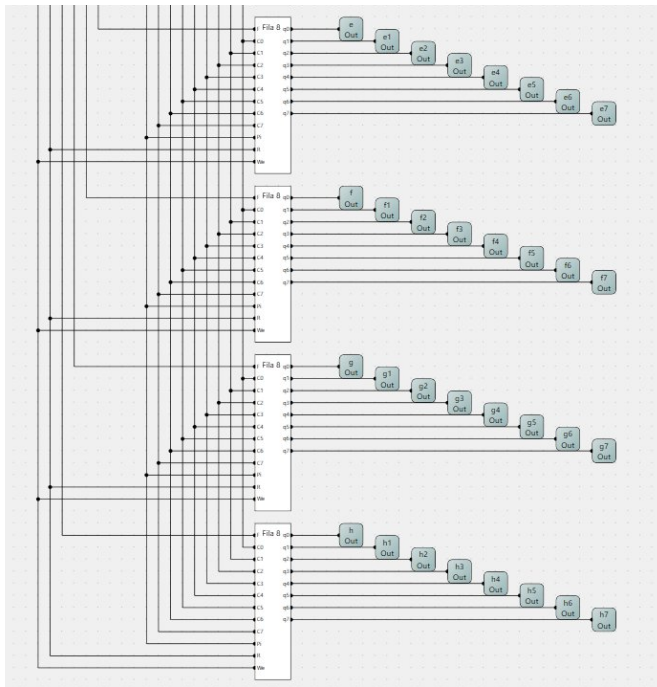


Este controlador tiene la función de conectar 8 celdas para formar una de las 8 filas de la matriz 8x8. Como se puede ver, todas las celdas comparten la misma entrada de fila ya que pues este controlador representa una sola fila, así que deben de responder a la misma señal.

Comparten también las entradas globales (por llamarlo de alguna manera) de enable, pixel y reset. Hay una entrada columna desde C0 hasta C7 ya que cada celda individual controla un pixel diferente en la misma fila. Por eso comparten fila y tienen columnas separadas, así cada uno de estos controladores maneja una fila de la forma [0,1,2,3,4,5,6,7], siendo cada elemento un pixel independiente.

Pantalla 8x8:





El circuito encargado de controlar la matriz integra 8 controladores de fila, controlando así las 64 celdas individuales por medio de estos. Utiliza decoders para emitir las señales. Uno para las filas y otro para las columnas. Las salidas del decoder de filas van cada una a un controlador de fila diferente (D0 va al controlador de la fila 0, D1 al de fila 1 y así sucesivamente). Y las salidas del de columnas van todas a ese espacio en ese decoder (D0 del decoder de columnas va a la entrada C0 de todos los controladores, D1 va a la entrada C1 de todos los controladores, y así sucesivamente) Estos decoders reciben coordenadas básicamente. Estas vienen por las entradas row y col, que representan la coordenada del pixel a encender. Por ejemplo, si entran unas coordenadas al decoder de filas para encender la fila 5 (101), la señal ingresa al módulo que controla la fila 5. Y si en el de columnas ingresa 4 (100), esa señal ingresará a todas las celdas, pero solo encenderá aquella que también reciba la señal 1 de filas. Esto encenderá la posición 5,4 de la matriz.

Distribución de módulos de trabajo:

Integrante	Lo que hizo para el Avance	Lo que hará para la Entrega Final
Johnny Chavarría	• Módulo de Pantalla 8x8: - Matriz con flip-flops para guardar los puntos encendidos. - Decodificadores de 3 a 8 para seleccionar fila y columna. - Circuito para escribir un punto con sus coordenadas (X,Y). • Junto con Andrés: Sumador y restador de 8 bits.	• Mejoras de la Pantalla: - Hacer la pantalla más grande (16x16 o 32x32). - Circuito para centrar los ejes X y Y (para que el (0,0) quede en el puro centro de la pantalla). - Botón para borrar toda la pantalla. - Selector para elegir qué función matemática se quiere dibujar en pantalla.
Andrés Cortés	• Multiplicador 3:8: - Multiplicador combinacional usando compuertas AND. - Sumadores conectados para sumar los productos y sacar el resultado de 8 bits. • Junto con Johnny: Sumador y restador de 8 bits.	• Cálculo de funciones y Control: - Circuito para calcular la parábola (multiplicar $X \cdot X$). - Contador automático para que la X vaya avanzando sola y dibuje la gráfica punto a punto. - Conectar displays de 7 segmentos para mostrar los valores en números normales. - Mejorar el multiplicador si el profe pide la versión secuencial.
Trabajo en equipo (Johnny y Andrés)	• Diseño y pruebas del Sumador-Restador de 8 bits en complemento a 2 y las salidas para saber si dio cero o negativo.	• Unir todo el proyecto: Conectar la pantalla, el reloj (reloj de pulsos), la calculadora y los botones en el circuito principal para que funcione en conjunto.